Tutorial 6 - Introduction to Node.js Servers Logbook

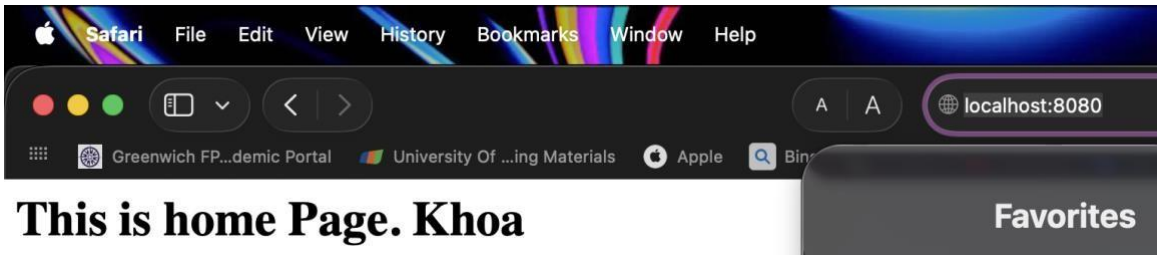
Name: Le Anh Khoa
ID: GCS230282

Screenshot 1: Running the Server

```
(base) mk183@Khoadzai tutorial6 % node server.js
Node.js web server at port 8080 is running... Khoa
```

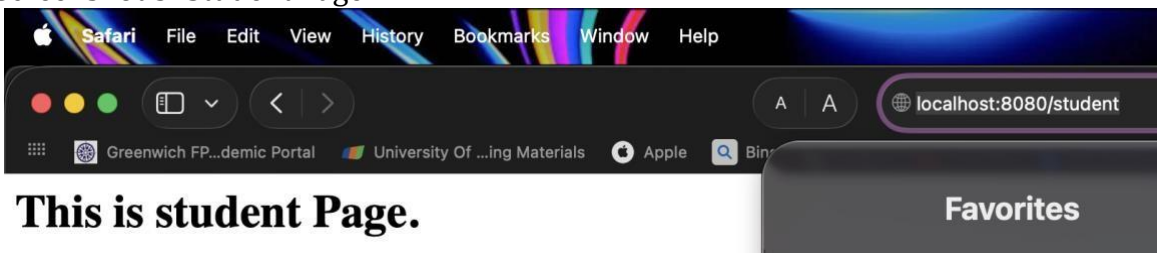
I learned how to initialize and run a basic Node.js web server using the node server.js command. By observing the terminal output, I understood that the server continuously listens for incoming client requests on a designated port (in this case, 8080). I also realized that using console.log() to output custom messages (like my name) is a practical way to verify that the correct script is executing and the server is actively running.

Screenshot 2: Home Page (Root URL)



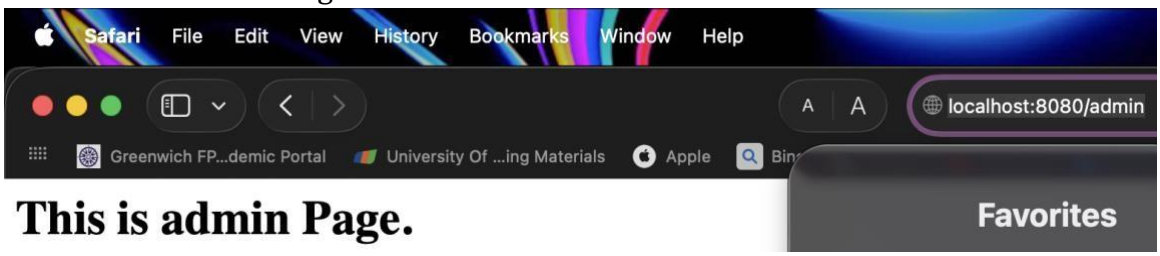
This step helped me grasp how a Node.js server handles a basic HTTP request to the root URL (/). I understand that by accessing the req.url property, the server can identify the exact path requested by the client. Furthermore, I learned the basics of constructing an HTTP response by defining a 200 (OK) status code to indicate success, followed by rendering and sending simple HTML content back to the browser.

Screenshot 3: Student Page



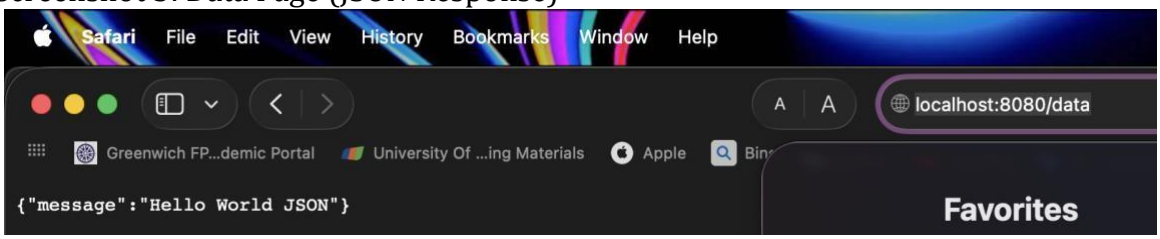
I practically explored the concept of server-side routing. I realized that by implementing simple if/else conditional logic based on the req.url value (such as /student), I can instruct the server to return completely different HTML templates depending on the specific path the user navigates to.

Screenshot 4: Admin Page



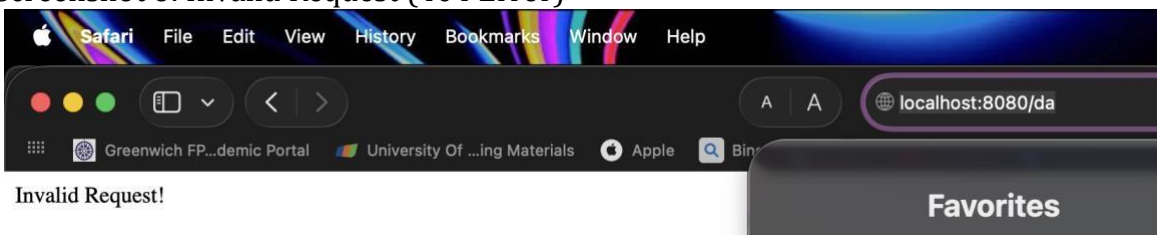
Building upon the previous step, setting up the /admin route solidified my understanding of how routing works. It demonstrated that a single Node.js JavaScript file can effectively act as a central dispatcher, capable of managing, distinguishing, and responding to multiple distinct route requests within the same application structure.

Screenshot 5: Data Page (JSON Response)



Through this route, I learned how to serve API-like endpoints by returning data in JSON format instead of standard HTML. I now understand the critical role of HTTP Response Headers; specifically, setting the Content-Type to application/json is necessary to inform the client how to parse the incoming data. Additionally, I grasped the use of JSON.stringify() to serialize a native JavaScript object into a transferable string format before sending it over the network.

Screenshot 6: Invalid Request (404 Error)



This task taught me how to handle undefined routes and implement basic server error handling. I learned that using a final, fallback else condition allows the server to catch any URLs that do not match my predefined routes. By returning a 404 (Not Found) HTTP status code alongside an "Invalid Request!" message, I understand how to properly communicate to the client's browser that the requested resource does not exist on the server.